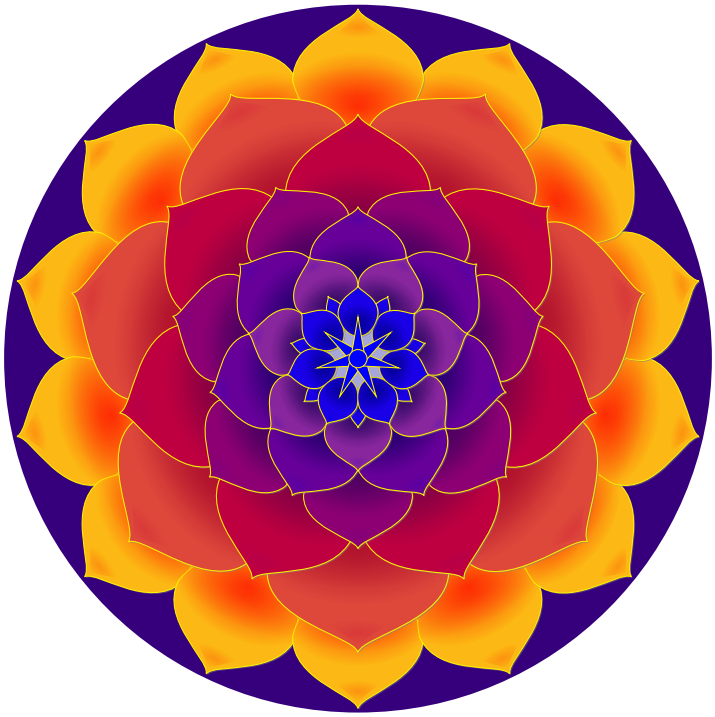


SandpileExperiment

Mathematics With Computer Science

Author: Zewan LYU(999019888)

Date: 2026-06-28

Contents

Chapter 1

Basic Setting

1.1 The Abelian Sandpile Model

The Abelian sandpile model is a cellular automaton introduced by Bak, Tang and Wiesenfeld. A d-dimensional lattice holds a non-negative integer $h(x)$ at each site x, the number of grains. A site is *stable* when $h(x) < T$, where T is the toppling threshold; it is *unstable* when $h(x) \geq T$. An unstable site *topples*: it loses T grains and sends one grain to each of its $2d$ nearest neighbours. Grains that leave the boundary are lost (open boundary condition). Repeated toppling eventually returns every site to a stable configuration; this final configuration is independent of the order of topplings (the Abelian property).

In two dimensions the grid is $N \times N$, the threshold is $T = 4$, and each site has four orthogonal neighbours. In three dimensions the cube is $n \times n \times n$, the threshold is $T = 6$, and each site has six neighbours $(\pm x, \pm y, \pm z)$.

1.2 Implementation of the Sandpile

The model is implemented in Python with NumPy. Two classes, `Sandpile2D` and `Sandpile3D`, share the same structure; we describe the 2D case for brevity. The grid is stored as a NumPy integer array. Stabilisation does not rescan the whole grid each step; instead it keeps a queue of currently unstable sites, so the running time is $O(N^2 + \text{number of topples})$. The core routine is:

Listing 1.1: Stabilisation by queue (2D, simplified).

```
def stabilize(grid, N, T=4):
    queue = deque(sites where grid >= T)
    topples = 0
    while queue:
        (x, y) = queue.popleft()
        if grid[x, y] < T:        # already stabilised by a neighbour
            continue
        grid[x, y] -= T           # topple
        topples += 1
        for (nx, ny) in neighbours(x, y):   # up, down, left, right
            if inside(nx, ny, N):
                grid[nx, ny] += 1
                if grid[nx, ny] >= T:
                    queue.append((nx, ny))
    return grid, topples
```

Each toppling event removes T grains from a site and distributes one grain to every in-bounds neighbour; any grain crossing the boundary is discarded. The 3D version is identical except that the threshold is 6 and each toppling distributes to six neighbours instead of four.

1.3 Experiment Design

The experiments follow the instructions given in the email correspondence that initiated this project. The protocol for a single trial is:

1. Start with an $N \times N$ (2D) or $n \times n \times n$ (3D) grid.

2. Put a fixed *background* of grains on every site: 3 grains in 2D, 5 grains in 3D. The background is chosen one below the threshold, so the grid is stable but on the verge of toppling.

3. Select three random distinct sites and add one extra grain to each. At least one site now reaches the threshold and an avalanche begins.

4. Stabilise the grid with the routine of Section 1.2.

5. Save the final stable grid to disk and record statistics: total topplings, number of distinct toppled sites, the grain distribution, the seed, and the perturbed positions.

This trial is repeated many times (10 000 in 2D, 2 000 in 3D). Each trial uses an independent random seed $s_i = s_0 + i$, so the perturbation sites differ every trial while the experiment as a whole remains reproducible. A central configuration file (`config.py`) holds all parameters—grid size, threshold, background, number of perturbation sites, number of trials, seed and output directories—so that the whole experiment can be retuned from one place. The driver (`main.py`) reads these parameters, runs the batch, and saves grids and aggregate statistics incrementally so that progress survives an interruption.

After the batch completes, the same driver produces the classification study of Chapter 2: for each direction (n, m) with $0 \le n, m \le 7$ it extracts the height pattern far from the centre along that direction and arranges the resulting images in a table.

1.4 Visualisation

A separate module (`visualize.py`, built on Matplotlib) turns the saved grids into figures. Each grain count is mapped to a fixed colour (white for 0, moccasin for 1, sandy brown for 2, peru for 3, and a warm colour for higher counts), so that the height pattern is read directly as a coloured image. The module provides:

- single-grid heatmaps and side-by-side comparisons of several trials;
- an avalanche-size histogram over all trials;
- a toppling-frequency map, obtained by averaging many stabilised grids against the background, which reveals the directions that recur across trials;
- for 3D, a set of z-slices and a slice montage, since a 3D grid is viewed one 2D layer at a time;
- interactive viewers with trial and slice sliders, so the user can browse the thousands of saved grids by hand and look for recurring structures.

These tools were used both to spot the regularities reported in Chapter 2 and to produce the figures in this report.

Chapter 2

Classification of Directional Patterns

After stabilizing 3000 trials of 301×301 sandpile grids (each with background 3, plus 3 extra grains at random sites), we examine the pattern formed by cells whose height differs from 3. Far from the center, along a fixed rational direction (n, m), a periodic height pattern emerges. We classify these patterns for all directions with $0 \leq n, m \leq 7$.

2.1 General Description of Directional Patterns

When one examines a stabilised grid far from the centre, the set of sites whose height differs from the background 3 forms a collection of lines. Each line extends outward in a fixed direction and is composed of a minimal periodic unit of coloured cells that repeats many times along that direction, giving every observed direction its own characteristic stripe. All observed directions are rational—that is, the slope of every line is a rational number; no line with an irrational slope appeared in any of the 10 000 trials.

2.2 Classification Table for (n, m), $0 \leq n, m \leq 7$

Table 2.1 shows the directional pattern for each pair (n, m). Only seven distinct primitive (coprime) directions were observed across all 3000 trials. When a pair (n, m) reduces by $\gcd(n, m)$ to one of these seven primitive directions, the corresponding image is shown; otherwise the entry reads “not found.”

Table 2.1: Directional pattern classification for $0 \le n, m \le 7$. Rows index n, columns index m. Entries show the image of the reduced coprime direction.

$n\downarrow\ /\ m\rightarrow$	0	1	2	3	4	5	6	7
0	—							
1					not found	not found	not found	not found
2				not found		not found		not found
3			not found		not found	not found		not found
4		not found		not found		not found	not found	not found
5		not found	not found	not found	not found		not found	not found
6		not found			not found	not found		not found
7		not found	not found	not found	not found	not found	not found	

The seven observed directions are:

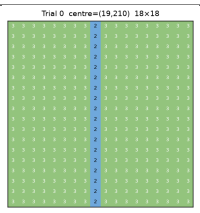
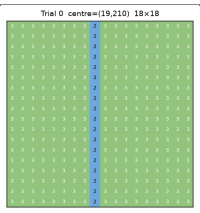
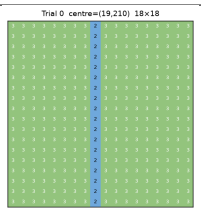
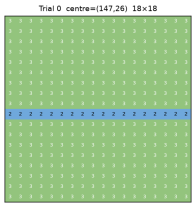
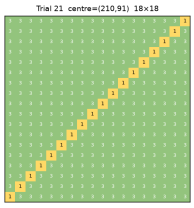
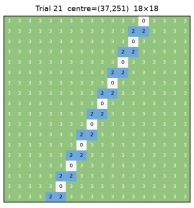
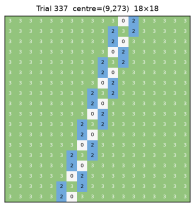
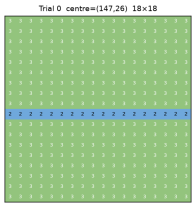
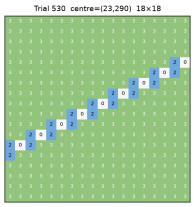
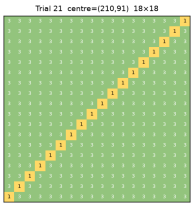
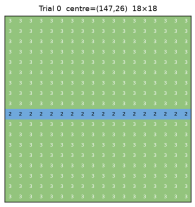
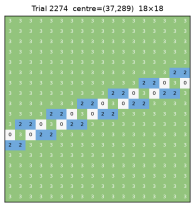
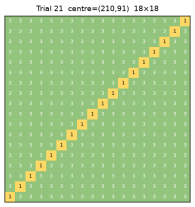
$$(0,1),\ (1,0),\ (1,1),\ (1,2),\ (1,3),\ (2,1),\ (3,1).$$

Note that (n,m) and (m,n) yield distinct patterns in general (e.g., $(1,2)$ vs. $(2,1)$ are different), and the negative directions $(-n,m)$, $(n,-m)$, $(-n,-m)$ produce the same patterns by symmetry. You can check the folder `images/` for the original images.

2.3 Enlarged Classification Table for (n,m), $0 \le n, m \le 3$

Table 2.2 shows the same classification restricted to $0 \le n, m \le 3$. Because the table is smaller, each image is displayed at a larger size so the periodic structure of every pattern is easier to inspect. The mapping from (n,m) to its reduced coprime direction is the same as in Table 2.1; entries that do not correspond to any observed primitive direction are marked "not found."

Table 2.2: Directional pattern classification for $0 \le n, m \le 3$ (enlarged). Rows index n, columns index m.

$n\downarrow\ /\ m\rightarrow$	0	1	2	3
0	—			
1				
2				not found
3			not found	

All seven primitive directions already appear within this $0 \le n, m \le 3$ range: $(0,1)$, $(1,0)$, $(1,1)$, $(1,2)$, $(1,3)$, $(2,1)$, and $(3,1)$. The only non-primitive pairs in this range are $(0,2) = (0,1)$, $(0,3) = (0,1)$, $(2,0) = (1,0)$, $(2,2) = (1,1)$, $(3,0) = (1,0)$, and $(3,3) = (1,1)$, each inheriting the image of its reduced direction. The two remaining pairs, $(2,3)$ and $(3,2)$, are coprime but were not observed in any trial.

Chapter 3

Observed Regularities

Beyond the classification, several quantitative regularities were noted across all trials.

3.0.1 Internal Color Periodicity

Along any directional line, the color pattern of heights repeats periodically without variation. For instance, a line with $(2, 1)$ always follows the sequence (brown, white, brown) from left to right.

3.0.2 Frequency of Occurrence

Lines whose slope is close to horizontal or vertical appear less frequently across trials. The vast majority of observed lines are horizontal, vertical, or at a 45° angle (slope ± 1). Lines with slopes $1/2$, $1/3$, 2, or 3 are comparatively rare.

3.0.3 Vector Sum at Intersections

When multiple directional lines meet at a single intersection point, the sum of their unit direction vectors equals zero. This suggests a form of local conservation governing the pattern geometry.